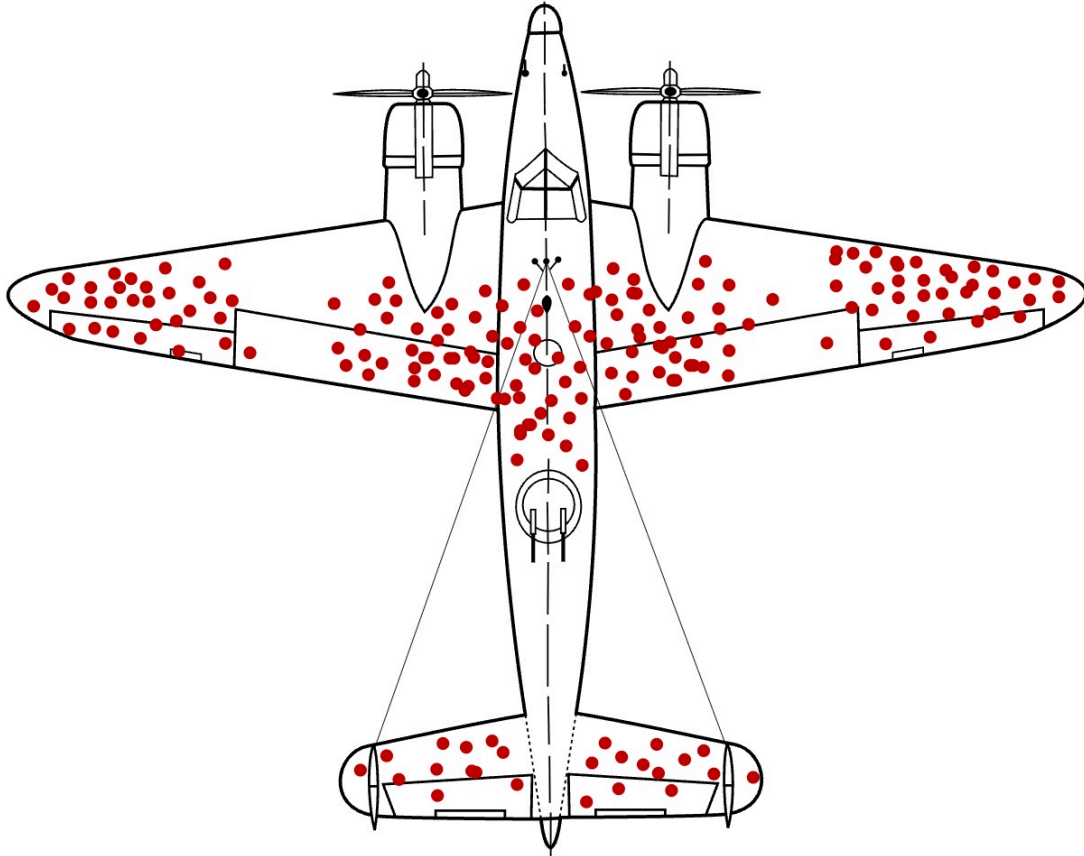


# Chart.js

---

Vom Endpunkt zur eigenen Grafik

# Wo würdet ihr die Flugzeuge verstärken?



Zweiter Weltkrieg: Die US-Armee untersucht Bomber, die von ihrem Einsatz zurückkommen.

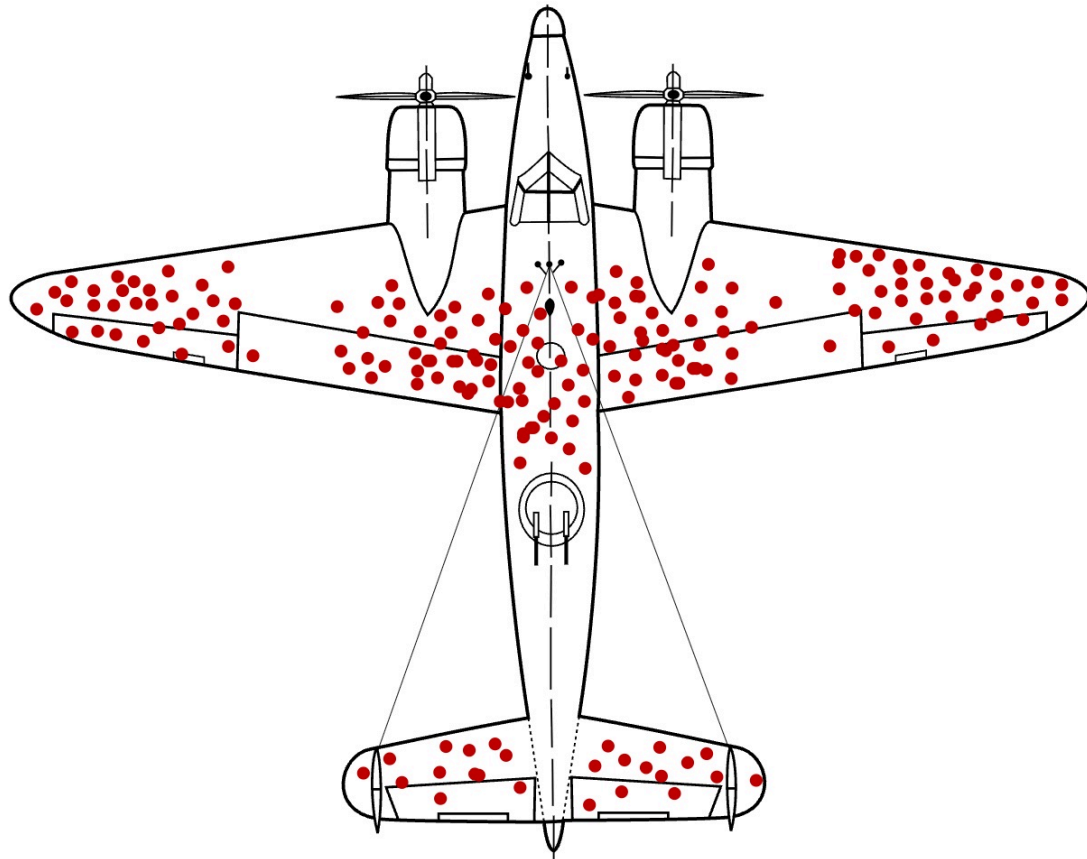
Jeder rote Punkt ist ein Einschussloch.

Panzerung ist schwer – ihr dürft nur wenige Stellen verstärken.

Zeigt auf die Stellen, die ihr verstärken würdet.

Hypothetisches Schadensbild nach Abraham Wald (1943). Illustration: Martin Grandjean, McGeddon, CC BY-SA 4.0, Wikimedia Commons.

# Die wichtigen Löcher fehlen



Motoren und Cockpit sind fast ohne Treffer.

Der Statistiker Abraham Wald fragte: Wo sind die Löcher der Flugzeuge, die nicht zurückkamen?

Wer am Motor oder im Cockpit getroffen wurde, stürzte ab.

Diese Flugzeuge fehlen in den Daten.

Verstärkt wird dort, wo keine Punkte sind.

# Was fehlt in euren Daten?

**Survivorship Bias** heisst: Wir sehen nur die Fälle, die einen Filter überstanden haben.

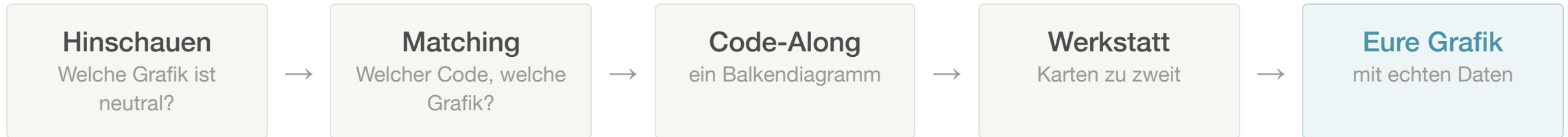
Der Filter ist oft unsichtbar:

- Bewertungen schreibt vor allem, wer sehr zufrieden oder sehr verärgert ist.
- Von Start-ups hören wir meist die erfolgreichen.
- Eine Umfrage per App erreicht nur Leute, die die App haben.

**Für euer Projekt:** Wer oder was fällt bei eurer Datenquelle durch den Filter?

Eine Grafik zeigt nur, was in den Daten steht – nicht, was fehlt.

# Was heute passiert



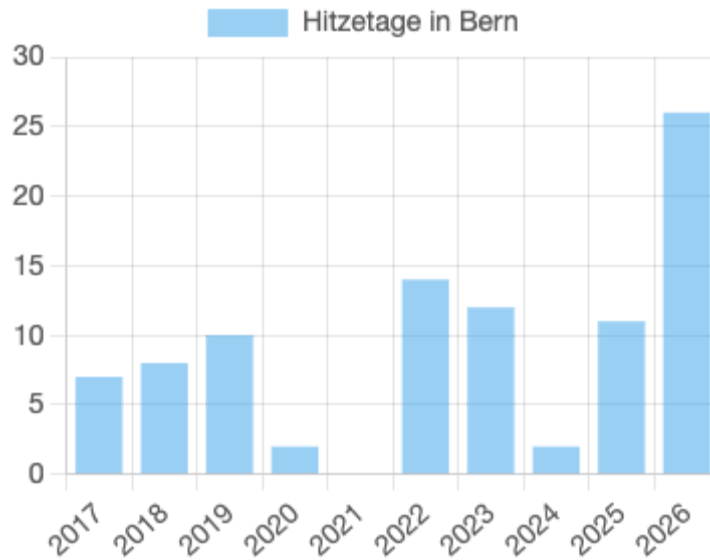
Am Ende des Tages zeigt eure Projektgrafik Daten aus eurem eigenen Endpunkt.

# Welche Grafik ist am neutralsten?

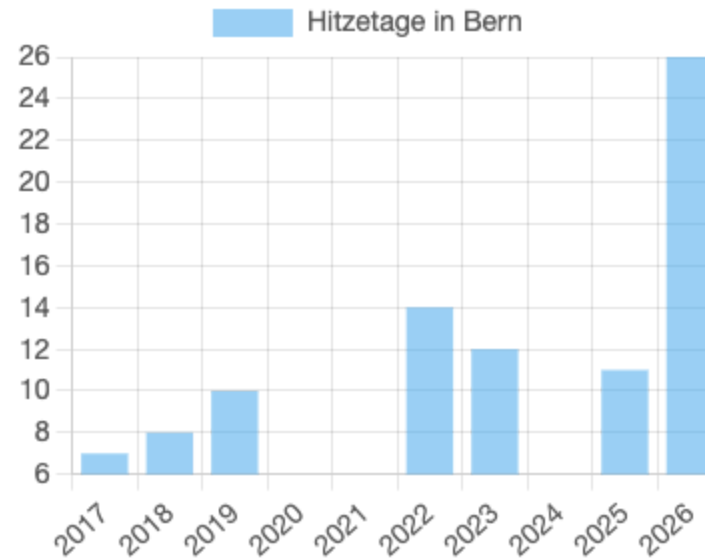
Dieselben Daten, drei Grafiken

# Hitzetage in Bern, 2017 bis 2026

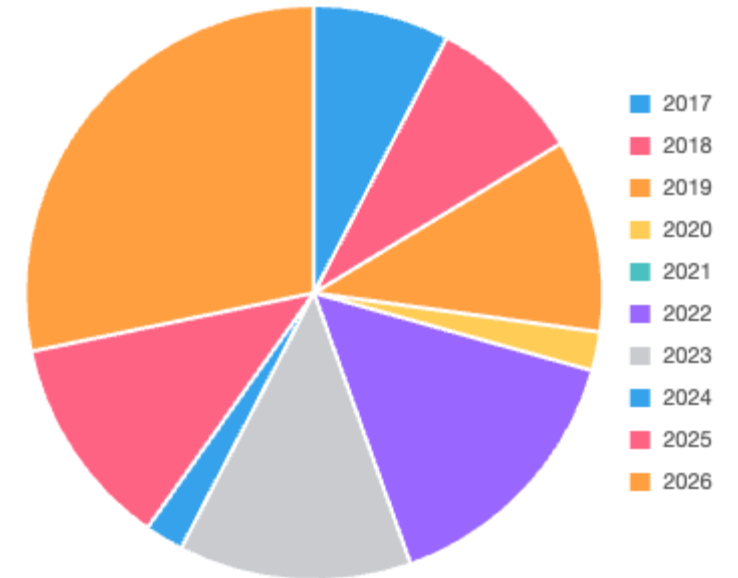
A



B



C



Stimmt mit der Hand ab: Welche Grafik zeigt die Daten am neutralsten – und warum?

# Der Unterschied ist jeweils eine Zeile

**A**

```
type: 'bar'
```

Die Achse beginnt bei null, die Länge der Balken entspricht dem Wert.

**B**

```
scales: { y: { min: 6 } }
```

Die Werte stimmen, aber drei Sommer verschwinden und die Unterschiede wirken grösser.

**C**

```
type: 'pie'
```

Jahre sind keine Anteile an einem Ganzen.

In Chart.js programmiert ihr Entscheidungen – und jede Entscheidung verändert die Aussage.



# Welcher Code zeichnet welche Grafik?

Zu zweit, zehn Minuten, ohne Laptop

# So geht das Matching

- Ihr bekommt acht Codekarten (A bis H) und acht Grafiken (1 bis 8).
- Ordnet zuerst die Codekarten A bis F zu.
- G und H sind Bonuskarten für schnelle Paare.
- Tragt bei jeder Grafik den Buchstaben ein.

Für A bis F gibt es zwei Grafiken zu viel – die letzte Karte ergibt sich also nicht von selbst.

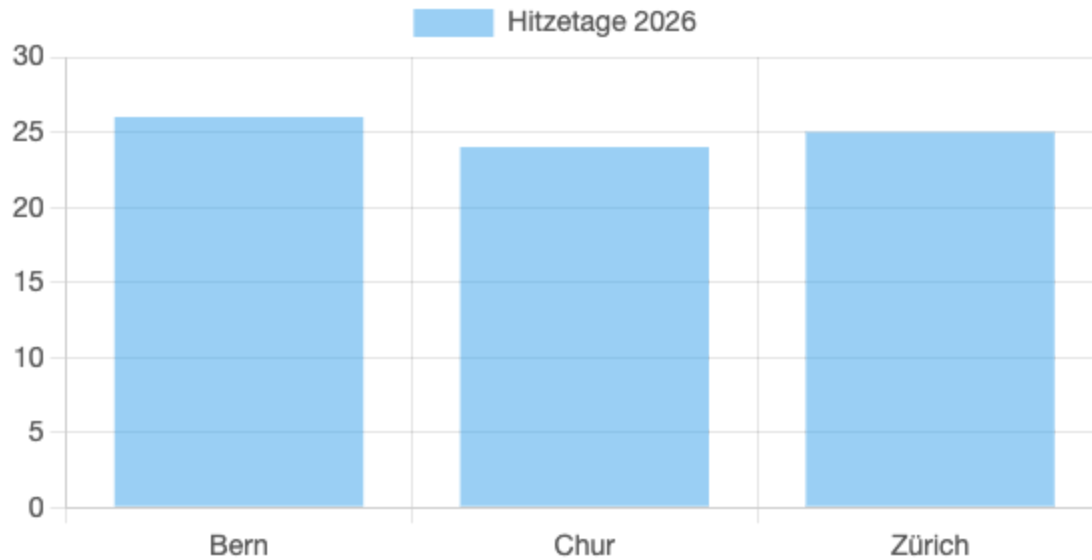
# Die Auflösung

| Code   | A | B | C | D | E | F | G | H |
|--------|---|---|---|---|---|---|---|---|
| Grafik | 4 | 8 | 2 | 6 | 1 | 7 | 5 | 3 |

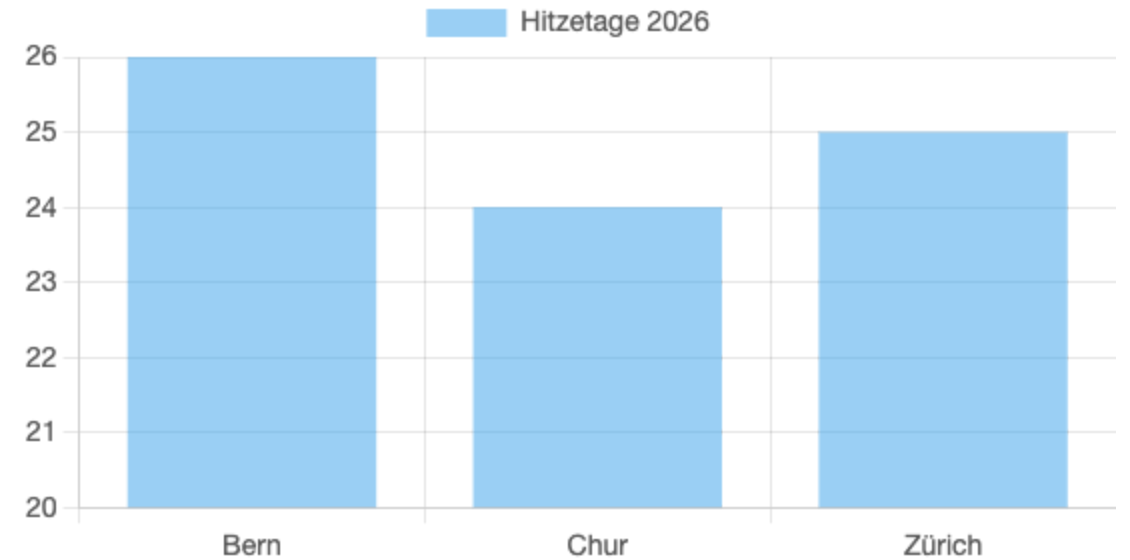
Bei welchem Paar wart ihr euch am wenigsten sicher – und welche Zeile hat entschieden?

# A und C zeigen dieselben Zahlen

A



C



```
options: { scales: { y: { min: 20 } } }
```

Bei Balken liest man den Wert an der Länge ab – deshalb beginnt die Achse bei null.

# Vom Endpunkt zum Balken

Ein kurzer Code-Along in drei Schritten

# Ein Diagramm braucht drei Dinge

IM HTML: DIE BIBLIOTHEK UND EINE ZEICHENFLÄCHE

```
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.5.1/dist/chart.umd.min.js"></script>
<canvas id="temperatur"></canvas>
```

IM JAVASCRIPT: DER AUFTRAG AN CHART.JS

```
new Chart(document.querySelector('#temperatur'), {
  type: 'bar', // welcher Diagrammtyp
  data: { labels: [], datasets: [] }, // was gezeigt wird
  options: {}, // wie es aussieht
});
```

# Chart.js will zwei Listen

## WAS DIE WETTER-API LIEFERT

```
"daily": {  
  "time": ["2026-10-08", "2026-10-09", ...],  
  "temperature_2m_max": [14.9, 14.7, ...]  
}
```

## WAS CHART.JS WILL

```
labels: ['2026-10-08', '2026-10-09', ...]  
data:   [14.9, 14.7, ...]
```

## DIE ZWEI LISTEN AUS DEM JSON HOLEN

```
const labels = json.daily.time;  
const data = json.daily.temperature_2m_max;
```

Beide Listen sind gleich lang und stehen in derselben Reihenfolge.

# Holen, umformen, zeichnen

```
async function zeichneTemperatur() {  
  // 1 Holen  
  const response = await fetch(ENDPUNKT);  
  const json = await response.json();  
  
  // 2 Umformen  
  const labels = json.daily.time;  
  const data = json.daily.temperature_2m_max;  
  
  // 3 Zeichnen  
  new Chart(document.querySelector('#temperatur'), {  
    type: 'bar',  
    data: {  
      labels: labels,  
      datasets: [{ label: 'Höchsttemperatur in Bern (°C)', data: data }],  
    },  
  });  
}
```



# Chart-Werkstatt

Elf Karten, ein Joker, zu zweit

# So spielt ihr

- Ihr arbeitet zu zweit an einem Laptop.
- Eine Person tippt, die andere liest Karte und Doku.
- Nach jeder Karte tauscht ihr die Rollen.
- Ihr wählt die Karten selbst – Stufe 1 vor Stufe 3.
- Gelöste Karten kommen an die Kartenwand.

## Erlaubt

- [chartjs.org/docs](https://chartjs.org/docs)
- Das Cheatsheet zu Chart.js
- AI mit «Erkläre mir ...»
- Nicht: AI mit «Schreib mir ...»

Stimmt euer Diagramm mit dem Zielbild auf der Karte überein, ist die Karte gelöst.

Zum Schluss ziehen alle die Lügen-Karte

Macht die Grafik so irreführend wie möglich – ohne einen einzigen Wert zu verändern.

Drei Paare zeigen ihre Grafik, die Klasse findet den Trick.

# Echte Daten, echte Fehler

Was passiert, wenn der Endpunkt nicht mitspielt

# Welche Ursache gehört zu welcher Meldung?

## MELDUNG IN DER KONSOLE

- A** blocked by CORS policy
- B** Unexpected token '<'
- C** Chart is not defined
- D** Canvas is already in use

## URSACHE

- 1** Auf derselben Zeichenfläche wird ein zweites Diagramm erzeugt.
- 2** Der Endpunkt erlaubt keine Anfragen von anderen Domains.
- 3** Die Antwort ist eine HTML-Seite statt JSON.
- 4** Die Bibliothek wurde nicht geladen.

A – 2 · B – 3 · C – 4 · D – 1

# Der Endpunkt muss andere Domains erlauben

Liegt die Grafik auf einer anderen Domain als der Endpunkt, blockiert der Browser die Antwort.

Eine Zeile im PHP-Endpunkt erlaubt den Zugriff:

```
header("Content-type: application/json");  
header("Access-Control-Allow-Origin: *");
```

**CORS** heisst Cross-Origin Resource Sharing: Der Server sagt, wer seine Daten abholen darf.

# Die Übergabe ist eine einzige Zeile

ZUERST: MOCK-DATEN NACH DEM DATENVERTRAG

```
const ENDPUNKT = 'data/mock.json';
```

SOBALD DAS BACKEND STEHT: DER EIGENE ENDPUNKT

```
const ENDPUNKT = 'https://eure-domain.ch/unload.php';
```

Funktioniert die Grafik danach unverändert, haben beide Teams den Datenvertrag eingehalten.

# Eure Grafik

Erst das Ticket, dann der Laptop



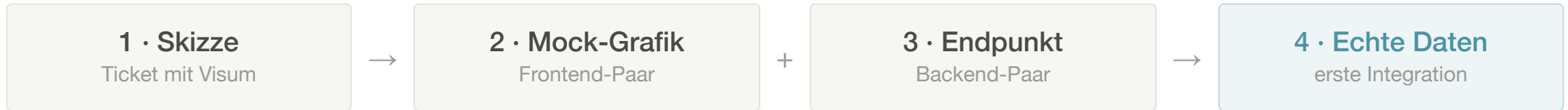
# Das Grafik-Ticket

Ihr füllt das Ticket zu viert aus, auf Papier:

1. Eure Aussage in einem Satz
2. Der Diagrammtyp und warum
3. Welches Feld aus dem Datenvertrag zu `labels` und `data` wird
4. Eine Skizze mit Titel und Achsentiteln
5. Der Lügen-Check

Mit dem Visum auf dem Ticket geht es an den Laptop.

# Wo steht eure Grafik?



Ihr verschiebt euren Punkt auf dem Board selbst.

Hängt ihr länger als 15 Minuten in einer Spalte, holt euch Hilfe.

# Speed-Galerie

- Je zwei Gruppen tauschen die Laptops.
- Ihr schaut die fremde Grafik eine Minute lang an.
- Dann sagt ihr ihre Aussage in einem Satz.

Stimmt euer Satz mit dem Satz auf ihrem Ticket überein, erzählt die Grafik, was sie soll.

# Zum Mitnehmen

Eine Grafik ist eine Aussage, keine Dekoration.

Zum Schluss: Exit Ticket ausfüllen